AHMADU BELLO UNIVERSITY DISTANCE LEARNING CENTRE

MASTERS IN INFORMATION MANAGEMENT (MIM)

Project Title	**Library Management System (Group Assignment)**
Course	LIBS-867 Database Management
Instructor	Shuaibu Muhammad
Group Members	- Aisha Muhammad Bamanga- P24DLLS82084 Group Leader - Abisola Tolulope Ogunyemi- P24DLLS82146 - Itohan Ugo -P24DLLS82301 - Shehu Adamu Hanafi -P24DLLS82670 - Ibrahim Ibrahim-P24DLLS80196 - Hassan - Abidemi Priscilla-P24DLLS82152 - Abiola Aluko - P24DLLS82824 - David Ajudua - P24DLLS82966 - Nanban Grace Dakhling-P24DLLS80902 - Abubakar Marwan
Institution	Distance Learning Centre, Ahmadu Bello University Zaria
Submission Date	05/12/2025

Executive Summary

This document presents a comprehensive database design for a multi-branch library management system implemented using MySQL 8.0+. While the system is specifically tailored for Nigerian library networks, supporting subject-based cataloging, multi-branch operations, member management, and integrated financial tracking through a unified payment infrastructure; the design is robust enough to be easily extended to support a global library establishment.

The database schema encompasses 17 interconnected tables supporting core functionalities including book cataloging, circulation management, member services, and financial operations. The implementation prioritises data integrity, scalability, and operational efficiency while addressing region-specific requirements.

The database for this assignment is hosting on a cloud MySQL instance and can be access for validation using the following credentials:

```
- Host: furthermore-mysql-furthermore.f.aivencloud.com
- User: group_four
- Password: Driver-Rear-Score7
- Port: 20426
- ItemSSL Mode: REQUIRED
```

NOTE: *This database instance will only be available for the lifespan of the course and will be taken down immediately afterwards*

The complete SQL script required to rebuild the database and the source file for this documentation has been committed a public github repository and can be accessed here:

```
https://github.com/linuxfreak/libs867_group_assignment.git
```

---

1. System Overview

1.1 Project Scope

The Library Index System is designed to manage operations across multiple library branches, with a focus on Nigerian and African literature. The system provides:

- **Multi-branch inventory management** across multiple branches in Lagos, Abuja, Kano, Port Harcourt, and Ibadan
- **Subject-based classification** supporting hierarchical categorisation
- **Comprehensive circulation tracking** with automated fine calculation
- **Integrated payment processing** for membership fees and fines
- **Membership management** with differentiated membership tiers
- **Publisher and author relationship tracking**

1.2 Technology Stack

- **Database Management System**: MySQL 8.0+
- **Storage Engine**: InnoDB (ACID-compliant with foreign key support)
- **Character Set**: UTF-8 (supporting international characters including Nigerian names)
- **Access Control**: Role-based permissions (the `group_four` user has SELECT, INSERT, UPDATE privileges)

1.3 Target Environment

The system is optimised for Nigerian library networks with considerations for:

- Local currency (Nigerian Naira - NGN)
- Regional phone number formats (15-digit international MSISDN format)
- Membership types (Standard, Student, Senior, Premium, Staff, Librarian)
- Local geographic hierarchy (State-based addressing)

---

2. Business Requirements & Use Cases

2.1 Core Business Requirements

BR-001: Multi-Branch Operations

Requirement: Support independent operations across geographically distributed library branches while maintaining centralized catalog management.

Implementation: Each branch maintains its own inventory (`book_copy` table) while sharing a common catalog (`book` table). This enables:

- Branch-specific availability tracking
- Inter-branch transfer capabilities
- Centralised acquisition reporting
- Localised member services

BR-002: Subject-Based Classification

Requirement: Organise books using a hierarchical subject taxonomy supporting African literature, Nigerian curriculum subjects, and international classifications.

Implementation: Self-referencing subject table with parent-child relationships enabling:

- Multi-level categorization (e.g., Literature → African Literature → Nigerian Literature)
- Cross-referencing through many-to-many relationships
- Primary subject designation for cataloging

BR-003: Circulation Management

Requirement: Track book borrowing with due date enforcement, renewal capabilities, and automated fine calculation.

Implementation: Comprehensive loan tracking system supporting:

- Multiple loan statuses (Active, Returned, Overdue, Lost)
- Renewal counting (maximum 5 renewals)
- Fine calculation with grace periods

BR-004: Financial Management

Requirement: Process membership fees and fines through a unified payment system with receipt generation.

Implementation: Multi-table payment architecture distinguishing:

- Membership payments (registration, annual, monthly, renewal)

- Fine payments (late returns, lost books)
- Payment status tracking (Pending, Completed, Failed, Refunded, Cancelled)
- Receipt generation with duplicate tracking

2.2 Primary Use Cases

UC-001: Member Book Search & Borrowing

Actor: Library Member
Precondition: Active membership
Flow:

1. Member searches catalog by title, author, subject, or ISBN
2. System displays available copies across branches
3. Member requests book at preferred branch
4. Librarian processes loan, updates availability status
5. System calculates due date based on membership type
6. Member receives loan confirmation with due date

Postcondition: Book marked as "Checked Out", loan record created

UC-002: Overdue Book Processing

Actor: Library System (Automated), Librarian
Precondition: Book past due date
Flow:

1. System identifies overdue loans daily
2. System calculates fines based on membership type and grace period
3. System updates loan status to "Overdue"
4. Librarian contacts member via stored contact information
5. Upon return, system calculates total fine
6. Librarian processes payment, updates loan status to "Returned"

Postcondition: Fine recorded, payment processed, book available for circulation

UC-003: Membership Registration

Actor: New Member, Librarian
Precondition: Valid identification
Flow:

1. Librarian collects member information
2. System assigns unique membership number
3. System determines applicable fees based on membership type

4. Member pays registration fee
5. System generates payment receipt
6. System creates member account with expiry date
7. Member receives membership card

Postcondition: Active member account created, payment recorded

UC-004: Book Acquisition & Cataloging

Actor: Library Acquisitions Staff
Precondition: Purchase approval
Flow:

1. Staff verifies book metadata (ISBN, title, author, publisher)
2. Staff creates book record if new title
3. Staff links authors and subjects
4. Staff creates book_copy record for each physical copy
5. System generates unique barcode
6. Staff assigns shelf location
7. Book added to branch inventory

Postcondition: Book available for circulation, inventory updated

---

3. Database Architecture

3.1 Architectural Pattern

The system implements a **normalized relational database** following Third Normal Form (3NF) principles with strategic denormalization for performance optimization. The architecture employs:

- **Entity tables**: Core business objects (book, member, branch, etc.)
- **Junction tables**: Many-to-many relationships (book_author, book_subject)
- **Lookup tables**: Enumerated values and fee structures (payment_type, fine_policy)
- **Transaction tables**: Business operations (loan, payment)
- **Audit capabilities**: Created_at and updated_at timestamps across all tables

3.2 Table Dependencies

Tier 1 (No Dependencies):

- publisher
- subject (self-referencing)
- branch
- author
- payment_type

Tier 2 (Single Dependencies):

- book → publisher
- member → branch
- membership_fee_structure (independent lookup)
- fine_policy (independent lookup)

Tier 3 (Multiple Dependencies):

- book_author → book, author
- book_subject → book, subject
- book_copy → book, branch

Tier 4 (Complex Dependencies):

- loan → book_copy, member, branch
- payment → member, payment_type, loan (optional), branch (optional)

Tier 5 (Payment Specializations):

- membership_payment → payment, member

- fine_payment → payment, loan
- payment_receipt → payment

---

4. Entity Relationship Model

4.1 Core Entities

Publisher

Purpose: Catalog publishers of books in the collection
Key Attributes: name, country, established_year, contact information
Relationships: 1:N with book (one publisher publishes many books)

Author

Purpose: Track book authors with biographical information
Key Attributes: first_name, last_name, birth_date, nationality
Relationships: M:N with book through book_author junction

Book

Purpose: Represent unique publications (not physical copies)
Key Attributes: ISBN, title, publication_year, format
Relationships:

- N:1 with publisher
- M:N with author through book_author
- M:N with subject through book_subject
- 1:N with book_copy

Subject

Purpose: Hierarchical classification system
Key Attributes: subject_code, name, parent_subject_id (self-referencing)
Relationships:

- Self-referencing for hierarchy
- M:N with book through book_subject

Branch

Purpose: Physical library locations
Key Attributes: branch_code, name, address, manager_name, seating_capacity
Relationships:

- 1:N with book_copy
- 1:N with member (home branch)
- 1:N with loan

Book Copy

Purpose: Individual physical copies of books
Key Attributes: barcode, acquisition_date, condition_status, availability_status, shelf_location
Relationships:

- N:1 with book
- N:1 with branch
- 1:N with loan

Member

Purpose: Library members/users
Key Attributes: membership_number, name, email, membership_type, expiry_date
Relationships:

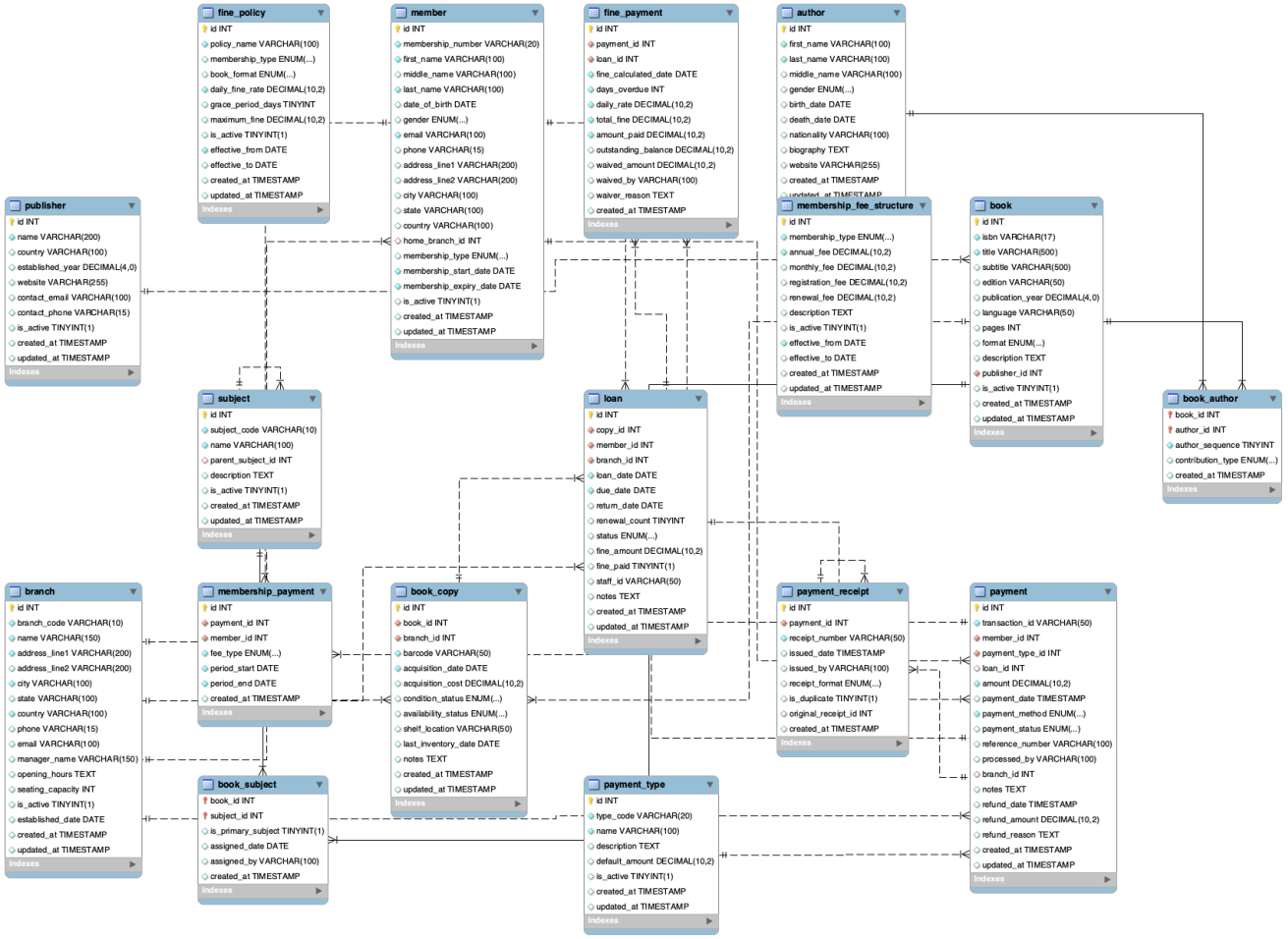
- N:1 with branch (home branch)
- 1:N with loan
- 1:N with payment

Loan

Purpose: Book borrowing transactions
Key Attributes: loan_date, due_date, return_date, status, fine_amount
Relationships:

- N:1 with book_copy
- N:1 with member
- N:1 with branch
- 1:1 with fine_payment (optional)

4.2 Entity Relationship Diagram



4.3 Cardinality Summary

Relationship	From	To	Cardinality	Type
Publisher-Book	Publisher	Book	1:N	One-to-Many
Author-Book	Author	Book	M:N	Many-to-Many
Book-Subject	Book	Subject	M:N	Many-to-Many
Book-BookCopy	Book	BookCopy	1:N	One-to-Many
Branch-BookCopy	Branch	BookCopy	1:N	One-to-Many
Branch-Member	Branch	Member	1:N	One-to-Many
BookCopy-Loan	BookCopy	Loan	1:N	One-to-Many
Member-Loan	Member	Loan	1:N	One-to-Many
Branch-Loan	Branch	Loan	1:N	One-to-Many
Loan-Payment	Loan	Payment	1:1	One-to-One (optional)
Member-Payment	Member	Payment	1:N	One-to-Many
Payment-Receipt	Payment	Receipt	1:1	One-to-One
Subject-Subject	Subject	Subject	1:N	Self-referencing

5. Schema Design & Implementation

5.1 Catalog Management Tables

5.1.1 Publisher Table

```
CREATE TABLE publisher (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200) NOT NULL UNIQUE,
    country VARCHAR(100),
    established_year DECIMAL(4,0) UNSIGNED,
    website VARCHAR(255),
    contact_email VARCHAR(100),
    contact_phone VARCHAR(15),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT year_above_867 CHECK (established_year >= 1450),
    CONSTRAINT chk_email_format CHECK (contact_email REGEXP '^[A-Za-z0-
9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$')
) ENGINE=InnoDB;
```

Design Notes:

- Uses DECIMAL(4,0) for established_year instead of YEAR datatype to support historical publishers before 1901
- Constraint enforces minimum year 1450 (Gutenberg printing press era)
- Email regex validation ensures data quality
- Soft deletion via is_active flag preserves referential integrity

5.1.2 Author Table

```
CREATE TABLE author (
    id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    gender ENUM('Male', 'Female', 'Not Provided', 'Other') DEFAULT 'Not
Provided',
    birth_date DATE,
    death_date DATE,
    nationality VARCHAR(100),
    biography TEXT,
    website VARCHAR(255),
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT chk_death_after_birth CHECK (death_date IS NULL OR
death_date >= birth_date)
) ENGINE=InnoDB;
```

Design Notes:

- Separate name fields support cultural naming conventions
- Gender field includes "Not Provided" default
- Death date validation prevents temporal inconsistencies
- TEXT field for biography supports detailed author information

5.1.3 Book Table

```
CREATE TABLE book (
    id INT AUTO_INCREMENT PRIMARY KEY,
    isbn VARCHAR(17) UNIQUE NOT NULL,
    title VARCHAR(500) NOT NULL,
    subtitle VARCHAR(500),
    edition VARCHAR(50),
    publication_year DECIMAL(4,0) UNSIGNED,
    language VARCHAR(50) DEFAULT 'English',
    pages INT UNSIGNED,
    format ENUM('Hardcover', 'Paperback', 'eBook', 'Audiobook',
'Magazine', 'Journal') DEFAULT 'Paperback',
    description TEXT,
    publisher_id INT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT fk_book_publisher FOREIGN KEY (publisher_id)
        REFERENCES publisher(id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_isbn_format CHECK (
        isbn REGEXP '^(97[89]-)?[0-9]{1,5}-[0-9]{1,7}-[0-9]{1,7}-[0-9X]$'
        OR isbn REGEXP '^[0-9]{9}[0-9X]$'
        OR isbn REGEXP '^[0-9]{13}$'
    ),
    CONSTRAINT chk_publication_year CHECK (publication_year >= 1450),
    CONSTRAINT chk_pages CHECK (pages > 0)
) ENGINE=InnoDB;
```

Design Notes:

- ISBN validation supports ISBN-10, ISBN-13, and hyphenated formats (https://en.wikipedia.org/wiki/ISBN)
- Extended title length (500 chars) accommodates lengthy academic titles
- FULLTEXT index on title, subtitle, description for efficient searching
- RESTRICT on publisher deletion prevents orphaned books

5.1.4 Subject Table (Hierarchical)

```
CREATE TABLE subject (
    id INT AUTO_INCREMENT PRIMARY KEY,
    subject_code VARCHAR(10) UNIQUE NOT NULL,
    name VARCHAR(100) UNIQUE NOT NULL,
    parent_subject_id INT NULL,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT fk_parent_subject FOREIGN KEY (parent_subject_id)
        REFERENCES subject(id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Design Notes:

- Self-referencing foreign key enables hierarchical taxonomy
- NULL parent_subject_id indicates top-level subjects
- SET NULL on parent deletion preserves child subjects
- subject_code provides human-readable identifiers (e.g., 'NIGLIT' for Nigerian Literature)

5.2 Junction Tables

5.2.1 Book_Author Table

```
CREATE TABLE book_author (
    book_id INT NOT NULL,
    author_id INT NOT NULL,
    author_sequence TINYINT UNSIGNED NOT NULL DEFAULT 1,
    contribution_type ENUM('Primary author', 'Co-author', 'Editor',
'Translator', 'Illustrator') DEFAULT 'Primary author',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (book_id, author_id),

    CONSTRAINT fk_ba_book FOREIGN KEY (book_id)
```

```
        REFERENCES book(id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ba_author FOREIGN KEY (author_id)
        REFERENCES author(id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_author_sequence CHECK (author_sequence > 0)
) ENGINE=InnoDB;
```

Design Notes:

- Composite primary key prevents duplicate author-book pairings
- author_sequence preserves author ordering for citations
- contribution_type distinguishes roles (primary, co-author, editor, etc.)
- CASCADE deletion maintains referential integrity

5.2.2 Book_Subject Table

```
CREATE TABLE book_subject (
    book_id INT NOT NULL,
    subject_id INT NOT NULL,
    is_primary_subject BOOLEAN DEFAULT FALSE,
    assigned_date DATE,
    assigned_by VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (book_id, subject_id),

    CONSTRAINT fk_bs_book FOREIGN KEY (book_id)
        REFERENCES book(id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_bs_subject FOREIGN KEY (subject_id)
        REFERENCES subject(id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Design Notes:

- is_primary_subject designates main classification
- assigned_by tracks cataloger for audit purposes
- Supports multiple subject assignments per book
- assigned_date enables temporal analysis of cataloging patterns

5.3 Inventory Management

5.3.1 Branch Table

```
CREATE TABLE branch (
    id INT AUTO_INCREMENT PRIMARY KEY,
    branch_code VARCHAR(10) UNIQUE NOT NULL,
    name VARCHAR(150) UNIQUE NOT NULL,
```

```
    address_line1 VARCHAR(200) NOT NULL,
    address_line2 VARCHAR(200),
    city VARCHAR(100) NOT NULL,
    state VARCHAR(100),
    country VARCHAR(100) NOT NULL DEFAULT 'Nigeria',
    phone VARCHAR(15),
    email VARCHAR(100),
    manager_name VARCHAR(150),
    opening_hours TEXT,
    seating_capacity INT,
    is_active BOOLEAN DEFAULT TRUE,
    established_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT chk_seating_capacity CHECK (seating_capacity >= 0),
    CONSTRAINT chk_email_branch CHECK (email REGEXP '^[A-Za-z0-
9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$')
) ENGINE=InnoDB;
```

Design Notes:

- branch_code provides short identifier for transactions (e.g., 'LAGOS01')
- Separate address fields support detailed location data
- seating_capacity supports capacity planning
- opening_hours as TEXT accommodates flexible schedule formats

5.3.2 Book_Copy Table

```
CREATE TABLE book_copy (
    id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT NOT NULL,
    branch_id INT NOT NULL,
    barcode VARCHAR(50) UNIQUE NOT NULL,
    acquisition_date DATE NOT NULL,
    acquisition_cost DECIMAL(10,2),
    condition_status ENUM('Excellent', 'Good', 'Fair', 'Poor', 'Damaged')
DEFAULT 'Excellent',
    availability_status ENUM('Available', 'Checked Out', 'Reserved', 'In
Repair', 'Lost', 'Withdrawn') DEFAULT 'Available',
    shelf_location VARCHAR(50),
    last_inventory_date DATE,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

```
    CONSTRAINT fk_copy_book FOREIGN KEY (book_id)
        REFERENCES book(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_copy_branch FOREIGN KEY (branch_id)
        REFERENCES branch(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_barcode_format CHECK (barcode REGEXP '^[0-9A-Z]
{8,20}$'),
    CONSTRAINT chk_acquisition_cost CHECK (acquisition_cost >= 0)
) ENGINE=InnoDB;
```

Design Notes:

- Represents physical copies, distinct from book (catalog) records
- barcode uniqueness enforced globally across all branches
- condition_status tracks physical wear
- availability_status drives circulation logic
- RESTRICT on deletion prevents inventory loss
- shelf_location supports efficient retrieval

5.4 Member Management

5.4.1 Member Table

```
CREATE TABLE member (
    id INT AUTO_INCREMENT PRIMARY KEY,
    membership_number VARCHAR(20) UNIQUE NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    last_name VARCHAR(100) NOT NULL,
    date_of_birth DATE,
    gender ENUM('Male', 'Female', 'Not Provided', 'Other') DEFAULT 'Not
Provided',
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    address_line1 VARCHAR(200),
    address_line2 VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(100) DEFAULT 'Lagos State',
    country VARCHAR(100) DEFAULT 'Nigeria',
    home_branch_id INT,
    membership_type ENUM('Standard', 'Student', 'Senior', 'Premium',
'Staff', 'Librarian') DEFAULT 'Standard',
    membership_start_date DATE NOT NULL,
    membership_expiry_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

```
    CONSTRAINT fk_member_branch FOREIGN KEY (home_branch_id)
        REFERENCES branch(id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_member_email CHECK (email REGEXP '^[A-Za-z0-
9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$'),
    CONSTRAINT chk_membership_dates CHECK (membership_expiry_date >
membership_start_date)
) ENGINE=InnoDB;
```

Design Notes:

- membership_number serves as human-readable identifier
- membership_type determines borrowing privileges and fees
- Email required for digital communication
- home_branch_id optional (SET NULL) - members can register without branch preference
- Expiry date validation prevents invalid membership periods

5.5 Circulation Management

5.5.1 Loan Table

```
CREATE TABLE loan (
    id INT AUTO_INCREMENT PRIMARY KEY,
    copy_id INT NOT NULL,
    member_id INT NOT NULL,
    branch_id INT NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE,
    renewal_count TINYINT DEFAULT 0,
    status ENUM('Active', 'Returned', 'Overdue', 'Lost') DEFAULT 'Active',
    fine_amount DECIMAL(10,2) DEFAULT 0.00,
    fine_paid BOOLEAN DEFAULT FALSE,
    staff_id VARCHAR(50),
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT fk_loan_copy FOREIGN KEY (copy_id)
        REFERENCES book_copy(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_loan_member FOREIGN KEY (member_id)
        REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_loan_branch FOREIGN KEY (branch_id)
        REFERENCES branch(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_due_date CHECK (due_date > loan_date),
    CONSTRAINT chk_return_date CHECK (return_date IS NULL OR return_date
>= loan_date),
```

```
    CONSTRAINT chk_renewal_count CHECK (renewal_count >= 0 AND
renewal_count <= 5),
    CONSTRAINT chk_fine_amount CHECK (fine_amount >= 0)
) ENGINE=InnoDB;
```

Design Notes:

- Tracks individual borrowing transactions
- status field drives workflow (Active → Overdue → Returned)
- renewal_count capped at 5 prevents indefinite renewals
- NULL return_date indicates unreturned book
- fine_amount calculated based on overdue days
- staff_id tracks processing librarian for accountability

5.6 Financial Management Tables

5.6.1 Payment Type Table

```
CREATE TABLE payment_type (
    id INT AUTO_INCREMENT PRIMARY KEY,
    type_code VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    default_amount DECIMAL(10,2),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT chk_default_amount CHECK (default_amount >= 0)
) ENGINE=InnoDB;
```

Design Notes:

- Lookup table for payment categories
- type_code provides programmatic identifiers (e.g., 'MEM_ANNUAL', 'FINE_LATE')
- default_amount suggests standard fees but allows overrides

5.6.2 Payment Table (Core)

```
CREATE TABLE payment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    transaction_id VARCHAR(50) UNIQUE NOT NULL,
    member_id INT NOT NULL,
    payment_type_id INT NOT NULL,
    loan_id INT,
```

```
    amount DECIMAL(10,2) NOT NULL,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    payment_method ENUM('Cash', 'Credit Card', 'Debit Card', 'Mobile
Money', 'Bank Transfer', 'Cheque', 'Online Payment') NOT NULL,
    payment_status ENUM('Pending', 'Completed', 'Failed', 'Refunded',
'Cancelled') DEFAULT 'Pending',
    reference_number VARCHAR(100),
    processed_by VARCHAR(100),
    branch_id INT,
    notes TEXT,
    refund_date TIMESTAMP,
    refund_amount DECIMAL(10,2),
    refund_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    CONSTRAINT fk_payment_member FOREIGN KEY (member_id)
        REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_payment_type FOREIGN KEY (payment_type_id)
        REFERENCES payment_type(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_payment_loan FOREIGN KEY (loan_id)
        REFERENCES loan(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_payment_branch FOREIGN KEY (branch_id)
        REFERENCES branch(id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_payment_amount CHECK (amount >= 0),
    CONSTRAINT chk_refund_amount CHECK (refund_amount IS NULL OR
(refund_amount >= 0 AND refund_amount <= amount))
) ENGINE=InnoDB;
```

Design Notes:

- Central payment repository for all financial transactions
- loan_id NULL for membership payments, populated for fine payments
- payment_method includes Nigerian-specific options (Mobile Money)
- reference_number stores external payment gateway IDs
- Refund tracking built into main table
- RESTRICT on member/loan deletion prevents financial record loss

5.6.3 Fine Payment Table

```
CREATE TABLE fine_payment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    payment_id INT NOT NULL,
    loan_id INT NOT NULL,
    fine_calculated_date DATE NOT NULL,
    days_overdue INT NOT NULL,
    daily_rate DECIMAL(10,2) NOT NULL,
```

```
    total_fine DECIMAL(10,2) NOT NULL,
    amount_paid DECIMAL(10,2) NOT NULL,
    outstanding_balance DECIMAL(10,2) DEFAULT 0.00,
    waived_amount DECIMAL(10,2) DEFAULT 0.00,
    waived_by VARCHAR(100),
    waiver_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_fp_payment FOREIGN KEY (payment_id)
        REFERENCES payment(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_fp_loan FOREIGN KEY (loan_id)
        REFERENCES loan(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_days_overdue CHECK (days_overdue >= 0),
    CONSTRAINT chk_daily_rate CHECK (daily_rate >= 0),
    CONSTRAINT chk_total_fine CHECK (total_fine >= 0),
    CONSTRAINT chk_amount_paid CHECK (amount_paid >= 0),
    CONSTRAINT chk_outstanding CHECK (outstanding_balance >= 0),
    CONSTRAINT chk_waived CHECK (waived_amount >= 0)
) ENGINE=InnoDB;
```

Design Notes:

- Specialization of payment table for fine details
- Tracks calculation methodology (days_overdue × daily_rate)
- Supports partial payments via outstanding_balance
- waiver functionality for discretionary forgiveness
- Audit trail via waived_by and waiver_reason

---

6. Constraints & Validation Rules

6.1 Data Integrity Constraints

6.1.1 Primary Key Constraints

All tables implement AUTO_INCREMENT integer primary keys for:

- Guaranteed uniqueness
- Efficient indexing
- Simplified foreign key relationships
- Consistent ID generation across distributed systems

6.1.2 Foreign Key Constraints

ON DELETE Actions:

- **RESTRICT**: Used for critical relationships preventing orphaned data
  - book_copy → book (prevent inventory without catalog)
  - loan → member, book_copy (preserve transaction history)
  - payment → member (maintain financial records)
- **CASCADE**: Used for dependent child records
  - book_author → book, author (remove relationships when entities deleted)
  - book_subject → book, subject (clean up classifications)
- **SET NULL**: Used for optional relationships
  - member → branch (home_branch_id) (allow member without branch assignment)
  - branch → payment (branch_id) (preserve payment if branch closed)

ON UPDATE Actions:

- All foreign keys use CASCADE to propagate identifier changes

6.1.3 Unique Constraints

- **Business Identifiers**: ISBN, membership_number, barcode, transaction_id
- **Name Fields**: publisher.name, branch.name (prevent duplicates)
- **Codes**: subject_code, branch_code (ensure unique identifiers)
- **Contact**: member.email (one account per email)

6.1.4 Check Constraints

Temporal Validation:

```
-- Birth before death
CHECK (death_date IS NULL OR death_date >= birth_date)
```

```
-- Due date after loan date
CHECK (due_date > loan_date)

-- Return date validity
CHECK (return_date IS NULL OR return_date >= loan_date)

-- Membership period validity
CHECK (membership_expiry_date > membership_start_date)
```

Business Rules:

```
-- ISBN format (ISBN-10, ISBN-13, hyphenated)
CHECK (isbn REGEXP '^(97[89]-)?[0-9]{1,5}-[0-9]{1,7}-[0-9]{1,7}-[0-9X]
       OR isbn REGEXP '^[0-9]{9}[0-9X]
       OR isbn REGEXP '^[0-9]{13})

-- Email format
CHECK (email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,})

-- Barcode alphanumeric 8-20 characters
CHECK (barcode REGEXP '^[0-9A-Z]{8,20})
```

Numeric Ranges:

```
-- Historical year validation (post-Gutenberg)
CHECK (established_year >= 1450)
CHECK (publication_year >= 1450)

-- Non-negative amounts
CHECK (acquisition_cost >= 0)
CHECK (fine_amount >= 0)
CHECK (seating_capacity >= 0)

-- Renewal limits
CHECK (renewal_count >= 0 AND renewal_count <= 5)

-- Page count
CHECK (pages > 0)
```

6.2 MySQL-Specific Constraint Limitations

Several constraints were documented but **not implemented** due to MySQL limitations:

6.2.1 Current Date Comparisons

```
-- DESIRED but NOT SUPPORTED:
CHECK (birth_date <= CURDATE())
CHECK (acquisition_date <= CURDATE())
CHECK (publication_year <= YEAR(CURDATE()) + 1)
```

Reason: MySQL CHECK constraints cannot reference non-deterministic functions like CURDATE() or CURRENT_TIMESTAMP.

Mitigation: Application-layer validation enforced during INSERT/UPDATE operations.

6.2.2 Cross-Table Validations

```
-- DESIRED but NOT SUPPORTED:
-- Prevent borrowing beyond member capacity
CHECK ((SELECT COUNT(*) FROM loan WHERE member_id = NEW.member_id AND
status = 'Active') <= 5)
```

Reason: CHECK constraints cannot contain subqueries.

Mitigation: Implemented through application logic or database triggers (not included in schema).

6.3 Data Quality Rules

6.3.1 Required vs Optional Fields

Always Required:

- Entity identifiers (names, codes, numbers)
- Transaction dates (loan_date, payment_date)
- Foreign keys for core relationships (book_id, member_id)
- Amounts for financial transactions

Optional (NULL allowed):

- Biographical details (birth_date, biography)
- Middle names
- Secondary addresses (address_line2)
- Death dates (for living authors)
- Return dates (for active loans)
- Reference numbers (for cash payments)

6.3.2 Default Values

Strategic defaults improve data consistency:

- `is_active = TRUE` (soft deletion pattern)
- `language = 'English'` (dominant language in collection)
- `format = 'Paperback'` (most common format)
- `country = 'Nigeria'` (primary service region)
- `renewal_count = 0` (new loans)
- `fine_amount = 0.00` (no fine initially)

---

7. Design Decisions & Rationale

7.1 Architectural Decisions

AD-001: Separation of Book and Book_Copy

Decision: Maintain separate tables for book (catalog) and book_copy (inventory).

Rationale:

- **Normalization**: Eliminates redundancy - title, author, publisher stored once
- **Multi-branch support**: Multiple copies of same book across branches
- **Independent lifecycle**: Catalog entry persists even if all copies retired
- **Efficient searching**: Catalog searches don't traverse inventory records

Trade-offs: Additional JOIN required for availability queries (mitigated by indexing).

AD-002: Hierarchical Subject Classification

Decision: Self-referencing subject table with parent_subject_id.

Rationale:

- **Flexibility**: Support multi-level taxonomies (e.g., Literature → African Literature → Nigerian Literature → Yoruba Literature)
- **Extensibility**: Easy to add new subject branches
- **Standards alignment**: Compatible with Library of Congress and Dewey classifications
- **Query capability**: Recursive CTEs can retrieve entire hierarchies

Alternative Considered: Flat subject list with naming conventions (e.g., "LIT_AFRICAN_NIGERIAN") - rejected due to rigidity.

AD-003: Composite Payment Architecture

Decision: Central payment table with specialized fine_payment and membership_payment tables.

Rationale:

- **Single source of truth**: All payments in one table
- **Type-specific details**: Specialization tables for additional attributes
- **Unified reporting**: Financial summaries query single table
- **Referential integrity**: Loan payments link to loan records

Alternative Considered: Separate fine and membership payment tables - rejected due to reporting complexity and code duplication.

AD-004: Enumerated Types vs Lookup Tables

Decision: Use ENUM for fixed, rarely-changing categories (gender, membership_type, format, payment_method, loan status).

Rationale:

- **Performance**: ENUM stored as integers (1-2 bytes)
- **Integrity**: Database enforces valid values
- **Code clarity**: Values explicit in schema
- **Simplicity**: No additional tables/JOINs

Trade-offs: Schema changes required to add values (acceptable for stable categories).

When Lookup Tables Used: payment_type, fine_policy, membership_fee_structure - these require additional metadata and change management.

AD-005: DECIMAL for Financial Amounts

Decision: Use DECIMAL(10,2) for all monetary values.

Rationale:

- **Precision**: Exact arithmetic (no floating-point errors)
- **Currency representation**: Two decimal places for Naira (NGN)
- **Range**: 10 digits accommodate ₦99,999,999.99 (sufficient for library operations)
- **Standards compliance**: Financial best practice

Alternative Rejected: FLOAT/DOUBLE - introduce rounding errors in calculations.

7.2 Indexing Strategy

7.2.1 Primary Indexes

All primary keys automatically indexed as B-tree (InnoDB default).

7.2.2 Foreign Key Indexes

Automatically created by InnoDB for foreign key columns to optimize JOIN operations.

7.2.3 Business Logic Indexes

```
-- Search optimization
CREATE INDEX idx_book_title ON book(title);
CREATE INDEX idx_author_last_name ON author(last_name, first_name);
CREATE INDEX idx_member_email ON member(email);

-- Filtering indexes
```

```sql
CREATE INDEX idx_book_publication_year ON book(publication_year);
CREATE INDEX idx_branch_active ON branch(is_active);
CREATE INDEX idx_copy_availability ON book_copy(availability_status);
CREATE INDEX idx_loan_status ON loan(status);

-- Composite indexes for common queries
CREATE INDEX idx_copy_book_branch ON book_copy(book_id, branch_id);
CREATE INDEX idx_loan_dates ON loan(loan_date, due_date);
CREATE INDEX idx_loan_overdue ON loan(due_date, status);

-- Full-text search
CREATE FULLTEXT INDEX idx_book_fulltext ON book(title, subtitle,
description);
```

Rationale:

- **Query patterns**: Indexes align with frequent WHERE, JOIN, and ORDER BY clauses
- **Composite indexes**: Cover multiple columns for specific query patterns
- **FULLTEXT**: Supports natural language book searches
- **Cardinality consideration**: High-cardinality columns (title, email) prioritized

7.3 Data Type Decisions

7.3.1 VARCHAR Lengths

Field	Length	Rationale
name (publisher, author)	100-200	Accommodate longest names
title, subtitle	500	Academic titles can be lengthy
email	100	RFC 5321 max 254, practical limit 100
phone	15	ITU-T E.164 maximum (international)
ISBN	17	ISBN-13 with hyphens: 978-0-306-40615-7
barcode	50	Future-proof for various schemes

7.3.2 Date vs Timestamp

- **DATE**: Used for business dates (loan_date, due_date, birth_date)
- **TIMESTAMP**: Used for audit trail (created_at, updated_at, payment_date)

Rationale: TIMESTAMP includes time component needed for transaction ordering; DATE sufficient for day-level granularity.

7.3.3 DECIMAL Precision

- **DECIMAL(4,0)**: Year values (0-9999)

- **DECIMAL(10,2)**: Currency amounts (up to ₦99,999,999.99)

7.4 Nigerian-Specific Adaptations

7.4.1 Phone Number Format

15-digit VARCHAR supports:

- Nigerian mobile: +234 (3) + 10 digits
- International format: +XXX (3) + up to 12 digits
- Domestic format: 0XXX (4) + 7 digits

7.4.2 Address Structure

```
address_line1 VARCHAR(200)
address_line2 VARCHAR(200)
city VARCHAR(100)
state VARCHAR(100)
country VARCHAR(100) DEFAULT 'Nigeria'
```

Accommodates Nigerian address conventions with state and city prominence.

7.4.3 Membership Types

```
ENUM('Standard', 'Student', 'Senior', 'Premium', 'Staff', 'Librarian')
```

Reflects Nigerian library membership structures with educational and institutional categories.

7.4.4 Payment Methods

```
ENUM('Cash', 'Credit Card', 'Debit Card', 'Mobile Money', 'Bank Transfer',
'Cheque', 'Online Payment')
```

Mobile Money inclusion reflects prevalent Nigerian payment ecosystem (e.g., Paga, OPay).

8.1 Data Population Summary

The database contains representative sample data reflecting a functional Nigerian library network. We started by seeding dummy data using the 'Dummy Data' service on: https://filldb.info/dummy/ and then refined the outputs until it was relevant enough for our purpose.

Table	Record Count	Coverage
publisher	15	Mix of international (Penguin, Oxford) and Nigerian publishers (Cassava Republic, Farafina)
author	20	Nigerian literary giants (Achebe, Soyinka, Adichie) plus contemporary authors
book	40	Nigerian/African literature, academic texts, history
subject	25	Hierarchical taxonomy including Nigerian literature sub-classifications
branch	13	Major Nigerian cities (Lagos, Abuja, Kano, Port Harcourt, Ibadan)
book_copy	68	Distributed inventory across branches
member	30	Diverse membership types and locations
loan	34	Mix of active, returned, and overdue loans
payment_type	7	Standard fee categories
fine_policy	6	Tiered policies by membership type

8.2 Test Scenarios

8.2.1 Search Functionality Tests

Test Case 1: Find all books by Chimamanda Ngozi Adichie

```
SELECT b.title, b.publication_year
FROM book b
JOIN book_author ba ON b.id = ba.book_id
JOIN author a ON ba.author_id = a.id
WHERE CONCAT(a.first_name, ' ', a.middle_name, ' ', a.last_name) =
'Chimamanda Ngozi Adichie'
ORDER BY b.publication_year;
```

Expected: 5 books (Purple Hibiscus, Half of a Yellow Sun, Americanah, We Should All Be Feminists, Dear Ijeawele)

Test Case 2: Find available copies of "Things Fall Apart" in Lagos

```sql
SELECT br.name, bc.barcode, bc.shelf_location
FROM book b
JOIN book_copy bc ON b.id = bc.book_id
JOIN branch br ON bc.branch_id = br.id
WHERE b.title = 'Things Fall Apart'
  AND br.city = 'Lagos'
  AND bc.availability_status = 'Available';
```

Expected: Multiple copies across Lagos Central, Ikeja, Victoria Island, Yaba branches

8.2.2 Business Logic Tests

Test Case 3: Verify overdue fine calculation

```sql
SELECT
    m.membership_number,
    b.title,
    l.due_date,
    DATEDIFF(CURDATE(), l.due_date) AS days_overdue,
    l.fine_amount,
    (DATEDIFF(CURDATE(), l.due_date) * 50) AS expected_fine
FROM loan l
JOIN member m ON l.member_id = m.id
JOIN book_copy bc ON l.copy_id = bc.id
JOIN book b ON bc.book_id = b.id
WHERE l.status = 'Overdue';
```

Validation: fine_amount should equal (days_overdue × daily_rate) based on membership type

Test Case 4: Verify membership expiry enforcement

```sql
SELECT membership_number, membership_expiry_date,
       DATEDIFF(membership_expiry_date, CURDATE()) AS days_until_expiry
FROM member
WHERE membership_expiry_date < DATE_ADD(CURDATE(), INTERVAL 30 DAY)
  AND is_active = TRUE
ORDER BY membership_expiry_date;
```

Expected: List of members requiring renewal notifications

8.2.3 Integrity Constraint Tests

Test Case 5: Attempt invalid ISBN

```sql
INSERT INTO book (isbn, title, publisher_id)
```

```
VALUES ('INVALID-ISBN', 'Test Book', 1);
```

Expected: ERROR 3819 - Check constraint 'chk_isbn_format' violated

Test Case 6: Attempt temporal violation

```
INSERT INTO loan (copy_id, member_id, branch_id, loan_date, due_date)
VALUES (1, 1, 1, '2024-11-20', '2024-11-15');
```

Expected: ERROR 3819 - Check constraint 'chk_due_date' violated

---

9. Future Enhancements

9.1 Feature Enhancements

FE-001: Digital Resource Management

Objective: Extend system to manage eBooks, audiobooks, and digital subscriptions.

Benefits:

- Unified catalog for physical and digital resources
- Automatic access expiration
- Usage analytics

FE-002: Reservation System

Objective: Allow members to reserve checked-out books.

Proposed Workflow:

1. Member places reservation on unavailable book
2. System queues request with priority_rank
3. On book return, system notifies first member in queue
4. Member has 48 hours to pick up (configurable expiry_date)
5. If not collected, next member notified

FE-003: Advanced Analytics Dashboard

Objective: Provide insights into library operations.

FE-004: Inter-Branch Transfer System

Objective: Enable book transfers between branches.

Proposed Workflow:

1. Branch requests transfer (e.g., high demand at receiving branch)
2. Approval workflow
3. Status tracking: Pending → Approved → In Transit → Completed
4. Automatic book_copy.branch_id update on completion

UX-003: Reading List Management

Objective: Allow members to curate personal reading lists.

---

10. Conclusion

10.1 Summary of Achievements

This library index system successfully addresses the core requirements of multi-branch library management through:

Robust Data Model:

- 17 interconnected tables supporting comprehensive library operations
- Normalised schema (3NF) ensuring data integrity and minimising redundancy
- Hierarchical subject organisation enabling flexible cataloging
- Unified payment architecture supporting diverse transaction types

Context Integration:

- Culturally relevant membership types and payment methods
- Support for Nigerian literature classification
- Localised address and phone number formats
- Multi-branch architecture

Scalability & Performance:

- Strategic indexing for common query patterns
- Foreign key constraints maintaining referential integrity
- Soft deletion patterns preserving historical data
- Foundation for partitioning and replication strategies

Financial Management:

- Integrated payment processing for fees and fines
- Tiered fine policies by membership type
- Complete audit trail for financial transactions
- Receipt generation and tracking

10.2 Key Design Strengths

1. **Separation of Concerns**: Clear boundaries between catalog (book), inventory (book_copy), and circulation (loan) enable independent lifecycle management
2. **Extensibility**: Self-referencing subject table, JSON columns for metadata, and modular payment architecture support future feature additions without schema restructuring
3. **Data Quality**: Comprehensive constraints (CHECK, UNIQUE, FOREIGN KEY) and validation rules prevent invalid data entry
4. **Audit Capability**: Timestamps on all tables, soft deletion flags, and detailed transaction logging support accountability and debugging

5. **Query Optimization**: Composite indexes, FULLTEXT search, and normalized structure balance read and write performance

10.3 Limitations & Trade-offs

Current Limitations:

- MySQL CHECK constraints cannot enforce dynamic validations (date comparisons with CURDATE())
- No built-in support for hierarchical queries (recursive CTEs have limited MySQL support)
- Fine calculation not automated (requires triggers or application logic)
- No versioning for catalog changes (book title updates lose history)

Accepted Trade-offs:

- ENUM over lookup tables: Better performance but schema changes for new values
- Denormalized member.home_branch_id: Improves query performance but requires manual updates
- Single audit timestamp: created_at/updated_at sufficient but not granular field-level tracking
- No document storage: Book covers, PDFs stored externally (file system or object storage)

10.4 Lessons Learned

Database Design:

- Balance normalization with query performance - not all 3NF patterns are optimal
- Index strategy matters more than schema complexity for performance
- Constraint enforcement at database level preferred over application-only validation
- Soft deletion crucial for financial and historical record preservation

Nigerian Context:

- Phone number field sizing must accommodate international formats
- State-based addressing more relevant than postal codes
- Membership tiers should reflect educational system structure

MySQL Specifics:

- DECIMAL for financial data is non-negotiable (avoid FLOAT/DOUBLE)
- FULLTEXT indexes powerful but require careful analyzer selection
- AUTO_INCREMENT safe for distributed systems with proper configuration

10.6 Final Remarks

This database design represents a production-ready foundation for library network operations. We have attempted balance academic rigor *(normalisation, constraints)* with practical considerations *(performance, maintainability).*

The modular architecture supports incremental enhancement - digital resources, reservation systems, and advanced analytics can be added without disrupting existing operations.

Future work should prioritise:

1. Implementation of proposed triggers for business logic automation
2. Integration with external APIs for catalog enrichment
3. Mobile application development leveraging the RESTful API layer
4. Advanced analytics for collection development decisions

References

1. MySQL Documentation: https://dev.mysql.com/doc/
2. Wikipedia: ISBN Format: https://en.wikipedia.org/wiki/ISBN
3. Database Third Norma Form (Normalisation): https://en.wikipedia.org/wiki/Third_normal_form
4.